

SQL Injection UNION Attack, Retrieving Multiple Values in a Single Column

Lab Overview

This lab demonstrates a **SQL Injection UNION attack** where multiple values need to be retrieved through a single column of the application's query result.

The vulnerability occurs because user-controlled input is incorporated into a SQL query without sufficient parameterization.

Objective

Exploit the SQL injection vulnerability to retrieve multiple values from the database through a single column.

Vulnerability

Vulnerability Type: SQL Injection

Attack Technique: UNION-based SQL Injection

The application is vulnerable because user-controlled input can modify the underlying SQL query.

Methodology

1. Open the vulnerable application.
2. Identify a parameter that influences a database query.
3. Test the parameter for SQL injection.
4. Determine the number of columns returned by the original query.
5. Identify which column can contain text.
6. Use the **UNION SELECT** technique to append a second query.
7. Retrieve multiple database values through the available text column.
8. Verify the returned information.

Exploitation

After determining the number of columns and identifying a suitable text-compatible column, a UNION query can be constructed.

A representative technique is:

```
' UNION SELECT NULL, username || ':' || password, NULL FROM users--
```

The exact syntax depends on the database management system and the number/order of columns in the original query.

The concatenation operation combines multiple database fields into a single returned value.

Result

The UNION query successfully retrieves multiple database values through a single column.

This demonstrates that an attacker can use SQL injection to extract information beyond the application's intended query.

Impact

SQL injection can potentially allow an attacker to:

- Read sensitive database information
- Extract credentials or other application data
- Modify database records
- Bypass application logic
- Perform destructive database operations depending on database privileges

Remediation

Developers should:

- Use parameterized queries / prepared statements.
- Avoid dynamically constructing SQL queries with untrusted input.
- Apply appropriate input validation as a secondary defense.
- Use least-privilege database accounts.
- Avoid exposing detailed database errors to users.

OWASP Reference

OWASP Top 10: A03 – Injection

Related vulnerability: SQL Injection

Conclusion

The lab demonstrates how UNION-based SQL injection can be used to retrieve multiple database values even when only one suitable output column is available. Parameterized queries are the primary defense against this class of vulnerability.

GitHub Commit Message

Add SQLi UNION multiple values report